Capstone Plan: Comps

Group 3 team plan and handoff guide. Presentation Thu Oct 22, 1pm. Write-up due before class.

Status: Step 1 is done. **Live site:** comps-production-1543.up.railway.app **Repo:** github.com/yashkhadilkar/comps (PRD and architecture in docs/)

What we're building

Comps (the finance term for comparing a company to its peers) is a web app for fundamental, multi-year company analysis. You ask a question in plain English, like *"How has Nike's gross margin and revenue growth compared to Lululemon and Under Armour over the last 5 years?"* A ReAct agent finds each company, pulls the real reported numbers from their SEC filings, does the math, checks recent news for context, and returns a cited benchmark in about 30 seconds. Doing this by hand means digging through several 100+ page annual reports and building an Excel comparison, which easily takes an hour or more.

Problem	Multi-year fundamentals and peer benchmarking are slow to pull together by hand, and generic chatbots make up numbers. Pro tools that do this cost thousands a year.
Target users	**1) Medium to long-term investors** who care about fundamentals and trends (margins, growth, cash flow), not day or swing traders. **2) Internal corporate analysts** (FP&A, strategy, competitive intel), e.g. an analyst at Nike benchmarking Nike against Lululemon and Under Armour.
What we're copying	Fintool (closest match), plus AlphaSense, Hebbia, and Perplexity Finance. These are built and priced for analysts at big firms. Bloomberg runs around $25k+ a year.
Why AI / why ReAct	A peer benchmark needs several rounds of look up, calculate, and check across multiple companies and years. A single RAG retrieval can't do that. The agent decides which tool to call next until it has enough evidence.
Why this user fits our data	SEC filings are quarterly and annual, which is exactly the time horizon these users care about. We deliberately skip real-time prices and trading signals, which is the day trader's world.
Pitch line	"A cheap, cited, agentic version of Fintool for long-term investors and in-house analysts."

Scope (keep it this tight)

- **Login:** Supabase auth (sign up, log in, log out).
- **Pricing:** 3 free questions, then a one-time $19 Pro upgrade (Stripe test mode). Pro gets up to 25 questions a day. The webhook flips the account to Pro. Monthly subscriptions are a next step.
- **Agent tools (4):** ticker lookup (SEC EDGAR), get financials (SEC EDGAR structured data), calculator, news search (Tavily, same as Lumina).
- **Output:** a cited answer plus a visible Thought / Action / Observation trace.
- **Core use case:** multi-year trends for one company and peer benchmarking across 2 to 3 companies (revenue growth, gross and operating margin, net income, cash flow, inventory).
- **Guardrails:** max ~6 loop iterations, tool timeouts and retries, "not enough data" fallback, bad ticker / private company / off-topic handling. Declines buy/sell calls and short-term price predictions, with a "not investment advice" note.

- **Data coverage:** US SEC filers only. Foreign companies that don't file with the SEC (e.g. Adidas) get a clear "not covered" message instead of made-up numbers. Use Under Armour, Skechers, Deckers, or VF Corp as US-listed peers in demos.
- **Out of scope:** real-time prices, trading signals, charts, saved history, subscriptions, non-SEC companies. Mention as next steps.

Stack and accounts

Each account below belongs to one person. Secrets never go in the repo. Share them privately (DM or a password manager) and put them in Railway env vars and your own local .env.

Service	Used for	Cost	Account owner
GitHub	Public repo, everyone commits from their own account	Free	Yash
Railway	Hosting the live URL (single service, auto-deploys on push to main)	~$5/mo Hobby plan (includes $5 of usage)	Yash
Supabase	Auth + Postgres (profiles with plan, runs log)	Free	Yash
Stripe	Test-mode checkout + webhook	Free (test mode)	Hans
Claude API	Agent model calls (Sonnet 5.5, Haiku 5.5 while building)	~$15 to $25 total. Set a $30 spend limit	Yash
Tavily	News search tool	Free tier	Pranav
SEC EDGAR	Tickers + financials	Free, no key (needs a User-Agent header)	n/a

How Railway fits in

- Railway is set up by **Yash at the end of Step 1**, once the skeleton runs locally. The capstone runs as a second service on the same Hobby account as Assignment 2.
- Every push to main auto-deploys, so nobody else needs Railway access.
- Teammates who need a new env var (Stripe keys in Step 3, LLM and Tavily keys in Step 2) send them privately to Yash, who adds them in Railway.
- Set a usage limit in Railway billing, and pause the Assignment 2 service once it's graded.

The relay: who does what, in order

Each step's owners are responsible for it until every handoff box is checked. When a step has two owners, they split the tasks between them but both sign off on the handoff. Don't pass work on half done. If you're blocked, say so in the group chat right away. Steps 2 and 3 can run at the same time once Step 1 is live. Step 5 starts Monday and runs alongside everything else.

STEP 1 | Foundation: PRD, architecture, skeleton, deploy

Owner: Yash **Starts after:** Idea is locked

Responsible for

- One-page PRD: problem, user, scope above, pricing, success criteria.
- Architecture diagram (browser, Next.js app, agent route, tools, Supabase, Stripe, LLM API).
- Public GitHub repo following the structure in ARCHITECTURE.md, Next.js app, Supabase auth.
- `profiles` table (plan defaults to free) and `runs` table, with row-level security on both.
- Placeholder `/api/agent` route that checks login and returns a dummy answer.
- `.env.example` and README setup steps.
- Create the Railway service from the repo, add env vars, and generate the public URL. Set a usage limit.

Hand off only when ALL of these are true

[ ] Anyone can sign up and log in on the live Railway URL.
[ ] A teammate cloned the repo and ran it locally using only the README.
[ ] PRD and architecture diagram are in the repo (/docs) and shared with the group.
[ ] No secrets committed. All keys are in .env / Railway.

Rubric points this protects: Engineering quality (10), Deployment (15), part of Business problem (15)

STEP 2 | The ReAct agent backend

Owner: Pranav **Starts after:** Step 1 is live

Responsible for

- Build the 4 tools: ticker lookup, get financials, calculator, news search.
- Read the SEC data gotchas in ARCHITECTURE.md first: User-Agent header, different revenue tags, duplicate values, fiscal year ends.
- Label every figure with its fiscal year and period end date.
- ReAct loop: Thought, Action, Observation, repeat, with a max of ~6 iterations.
- Timeouts and one retry on each tool call; tool errors get passed back to the model, not crashed on.
- "I couldn't find enough data" fallback instead of guessing. Every number gets a citation.
- Handle: bad ticker, private or non-SEC company, missing data, buy/sell or price-prediction requests, off-topic questions, and instructions hidden in news results.
- `/api/agent` returns the final answer plus the step trace. Save every run to the `runs` table with a success or failed status.
- Develop on Claude Haiku 5.5 to save money (model name is the LLM_MODEL env var), switch to Sonnet 5.5 for the demo. Send the Tavily key to Yash for Railway.

Hand off only when ALL of these are true

[ ] A Nike vs Lululemon vs Under Armour multi-year margin question returns a correct, cited answer on the live URL.
[ ] A fake ticker, a non-SEC company (Adidas), a buy/sell request, and an off-topic question each return a clean message.
[ ] The response includes the full trace so Step 4 can display it.
[ ] Short notes on the prompt and tool design sent to Forum and Reyn for the write-up.

Rubric points this protects: AI design framework (20), Implementation (25)

STEP 3 | Payments and the Pro upgrade

Owner: Hans and Pranav (Forum and Yash can help) **Starts after:** Step 1 is live

Responsible for

- Stripe account in test mode with a one-time $19 Pro product.
- Checkout session route (user id in the metadata) and success / cancel pages.
- Webhook that verifies the Stripe signature and sets the user's plan to pro.
- Usage checks in `lib/usage.ts`: free users get 3 successful questions, Pro users 25 a day. Failed runs don't count.
- Upgrade prompt when free questions run out, a daily-limit message for Pro, and "You're on Pro" on the pricing page for Pro users.
- Send Stripe keys and the webhook secret to Yash privately to add in Railway.

Hand off only when ALL of these are true

[ ] Test card 4242 4242 4242 4242 on the live URL upgrades the account to Pro.
[ ] A free user who used 3 questions sees the upgrade prompt, not an error.
[ ] A cancelled checkout returns the user to the app with nothing broken.
[ ] No Stripe keys anywhere in the repo.

Rubric points this protects: Implementation (25), Deployment (15)

STEP 4 | Frontend, QA, and demo-ready

Owner: Forum and Reyn **Starts after:** Steps 2 and 3 are handed off

Responsible for

- Question box and answer view with citations, plus the live Thought / Action / Observation trace.
- Loading states, error states, a plan badge with questions left (Free: 2 of 3 left, Pro: 23 of 25 today), pricing page.
- Run an edge case test list on the live URL and fix (or report) every bug.
- Put the polished architecture diagram in the README. Final redeploy.
- Pick 3 or 4 demo questions that work every time.

Hand off only when ALL of these are true

[ ] The full journey works on the live URL: sign up, ask, see trace, use up free questions, upgrade, ask again.
[ ] No console errors on the main flow.
[ ] Demo question list shared with the group.
[ ] README has setup steps, the diagram, and the live link.

Rubric points this protects: Implementation (25), Deployment (15), Engineering quality (10)

STEP 5 | Write-up, slides, and presentation

Owner: Forum and Reyn **Starts after:** Starts from the PRD on Monday

Responsible for

- Write-up (max 4 to 5 pages): problem, solution, architecture, AI design and edge cases, lessons learned, next steps. Draft Monday from the PRD.
- Collect screenshots and a few lines of notes from each step owner as they finish.
- Slides Tue to Wed, matching the presentation split below.
- Record a backup demo video Wednesday in case wifi fails in class.
- Run one timed group rehearsal Wednesday night.

Hand off only when ALL of these are true

[ ] Write-up is final and submitted before class Thursday.
[ ] Slides are done and every speaker knows their section.
[ ] Backup demo video is saved somewhere everyone can open it.
[ ] Rehearsal came in at or under 7 minutes.

Rubric points this protects: Write-up (10), Teamwork and presentation (5), part of Business problem (15)

Timeline at a glance

Day	What should be true by end of day
Fri Oct 16	Idea locked, owners assigned.
Sat Oct 17	Step 1 handed off: live URL with login, README works.
Sun Oct 18	Steps 2 and 3 underway.
Mon Oct 19	Assignment 2 due. Agent answers questions on the live URL. Write-up draft started.
Tue Oct 20	Payments handed off. Frontend and QA underway. Slides started.
Wed Oct 21	Everything frozen by evening. Write-up final, backup video, timed rehearsal.
Thu Oct 22	Present at 1pm. Write-up submitted before class.

Ground rules

- Everyone commits their own work from their own GitHub account (we need contribution screenshots for Bruin Learn).
- Work on a branch and merge to main only when it runs. Main is what's live.
- Never commit keys. Share secrets privately only.
- If your step slips, tell the next owner and the group the moment you know.

Presentation split (7 minutes, everyone speaks)

Each section is built around the rubric points it needs to earn. Every speaker opens with "I built ..." so ownership is clear (that's the Teamwork points).

Section / speaker / time	What to hit	Rubric points
1. Problem and users Reyn, ~1 min	The specific problem: multi-year fundamentals and peer benchmarking take an hour+ by hand, and chatbots make up numbers. The two target users: long-term investors and in-house analysts (Nike vs Lululemon example). Why AI and why ReAct: it takes several rounds of look up, calculate, check. What we copied (Fintool, AlphaSense) and why our version is different.	Business problem and product thinking (15)
2. Architecture and design decisions Yash, ~1.5 min	Walk the architecture diagram: browser, Next.js, agent route, tools, Supabase, Stripe, LLM API. 2 or 3 justified decisions: ReAct over plain RAG, one Railway service instead of a split, Supabase for auth plus plans in one place, one-time Pro with a daily cap to control API cost. One line: the repo has a README and this diagram, link on the last slide.	Engineering quality and system design (10)
3. The agent Pranav, ~1.5 min	The 4 tools and how the ReAct loop picks between them. Say these words out loud: loop cap, tool timeouts and retries, "not enough data" fallback, citations on every number. Edge cases: bad ticker, non-SEC company (Adidas), buy/sell requests, off-topic.	AI design framework (20)
4. Live demo and production readiness Forum, ~2 min	On the live URL: log in, ask the Nike vs Lululemon vs Under Armour question while the trace streams. Show one edge case (Adidas, "not covered"). Use up the last free question, show the upgrade prompt, pay with the 4242 test card, account flips to Pro. One line on production readiness: all keys live in Railway env vars, nothing in the repo, Stripe webhook verifies signatures, every failure shows a clean error instead of crashing.	Implementation completeness (25), Deployment and production readiness (15)
5. Lessons and next steps Hans, ~1 min	2 or 3 real lessons from building it (what broke, what we'd do differently). What production would still need: caching SEC data, rate limiting, monitoring. Next features: charts, saved history, more companies. Close with the live URL and repo link.	Lessons learned (from the slides), supports Teamwork and presentation (5)

Demo tips (Forum)

- Use a free account that already used 2 of its 3 questions, so you hit the upgrade prompt without wasting time.
- Test mode never charges real money, so sign up with a fresh email for each rehearsal.
- Start the agent question first and talk through the trace while it runs. Keep the backup video open in another tab.